

CO3-Assesment Tool-01

Name: R. Guruprasad Achari

Reg no:192472094

Course Code: Csa1214

Course: Computer architecture

Q1. Register Transfer and ALU Operations

Aim:

To design and simulate register transfer operations and basic arithmetic and logic operations using an ALU.

Registers: R1, R2, R3, R4

Operations:

1. $R1 \leftarrow R2$ — Transfer data from R2 to R1.
2. $R3 \leftarrow R1 + R2$ — Addition using ALU.
3. $R4 \leftarrow R3 - R1$ — Subtraction using ALU.
4. $R1 \leftarrow R2 \text{ AND } R3$ — Logical AND.
5. $R2 \leftarrow R3 \text{ OR } R4$ — Logical OR.

Role of ALU:

The ALU performs arithmetic and logical operations, while registers store and transfer the data.

Conclusion:

Register transfers and ALU operations were successfully simulated and the sequence of data movement was analyzed.

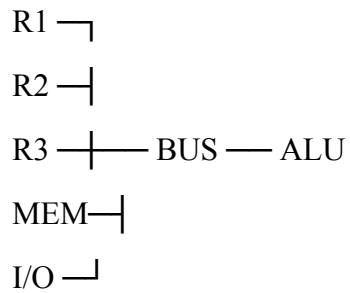
Q2. Single-Bus and Multi-Bus Organization

Aim:

To model single-bus and multi-bus computer organizations and compare their data-transfer efficiency.

Single-Bus

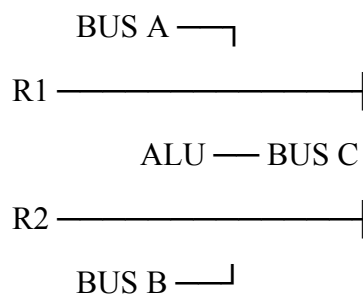
All registers, memory and I/O share one common bus.



Only one major transfer can occur at a time, causing bottlenecks.

Multi-Bus

Multiple buses allow simultaneous transfers.



Comparison

Feature	Single-Bus	Multi-Bus
Data transfer	Sequential	Parallel
Speed	Lower	Higher
Hardware cost	Low	Higher
Bottleneck	High	Low
Efficiency	Moderate	High

Conclusion:

Multi-bus organization improves performance by allowing multiple data transfers simultaneously.

Q3. Five-Stage Instruction Pipeline

Aim:

To simulate a 5-stage pipeline and compare its performance with a non-pipelined processor.

Pipeline Stages

IF → ID → EX → MEM → WB

- **IF:** Instruction Fetch

- **ID:** Instruction Decode
- **EX:** Execute
- **MEM:** Memory Access
- **WB:** Write Back

For 5 instructions:

Cycle: 1 2 3 4 5 6 7 8 9

I1: IF ID EX MEM WB

I2: IF ID EX MEM WB

I3: IF ID EX MEM WB

I4: IF ID EX MEM WB

I5: IF ID EX MEM WB

Performance:

For n instructions and k pipeline stages:

- Non-pipelined $\approx n \times k$ cycles
- Pipelined $\approx k + n - 1$ cycles

For 5 instructions and 5 stages:

- Non-pipelined = **25 cycles**
- Pipelined = **9 cycles**

Conclusion:

Pipelining increases instruction throughput and reduces the average execution time.

Q4. Pipeline Hazards

Aim:

To simulate data, control and structural hazards and apply techniques to resolve them.

Types of Hazards

1. Data Hazard

I1: ADD R1,R2,R3

I2: SUB R4,R1,R5

I2 requires the result produced by I1.

Solution: Data forwarding or stalling.

2. Control Hazard

Occurs due to branch instructions.

BEQ R1,R2,LABEL

Solution: Branch prediction.

3. Structural Hazard

Occurs when two instructions require the same hardware resource simultaneously.

Solution: Resource duplication or pipeline stalling.

Hazard Resolution

Hazard	Solution
Data	Forwarding / Stalling
Control	Branch Prediction
Structural	Extra resources / Stalling

Conclusion:

Hazard-handling techniques reduce pipeline delays and improve processor performance.

Q5. Superscalar, Out-of-Order and SMT Processor

Aim:

To analyze parallel instruction execution using superscalar, out-of-order, speculative execution and SMT.

Superscalar Execution

A processor executes **multiple instructions in one clock cycle**.

Instruction 1 —┐

Instruction 2 —┴—→ Multiple Execution Units

Instruction 3 —┘

Out-of-Order Execution

Instructions are executed when their required resources and data are available rather than strictly following program order.

Speculative Execution

The processor predicts future instructions, especially branches, and executes them before knowing the actual result.

Hyper-Threading / SMT

A single physical processor core handles multiple instruction threads simultaneously.

Advantages

- Higher instruction throughput
- Better CPU resource utilization
- Reduced idle execution units
- Improved multitasking performance

Comparison

Technique	Main Benefit
-----------	--------------

Superscalar	Multiple instructions/cycle
-------------	-----------------------------

Out-of-order	Reduces waiting
--------------	-----------------

Speculative	Reduces branch delay
-------------	----------------------

SMT	Executes multiple threads
-----	---------------------------

Conclusion:

Superscalar execution, out-of-order execution, speculation and SMT improve **instruction throughput and resource utilization**, resulting in better overall processor performance.